LAB 7: STATE-SPACE DESIGN FOR POSITION CONTROL IN A DC MOTOR

```
%--------------------------------------
% Fill in your student Name and ID.
Students.Name = 'Seyed Yazdan Zinati';
Students.ID = '260928077';
%--------------------------------------
```

0. Objectives

In this lab, we will experiment with different controller design to regulate the angular position of a DC Motor. For this we will use state-space representation in **MATLAB** and the function **lsim** to simulate the response of continuous linear systems to arbitrary inputs.

In section 1, we will define the DC Motor model. In section 2, we will design an output feedback controller for angular position control. In section 3, we will study the effect of model mismatch on the output feedback control. Finally, in section 4, we will design a state feedback with an integral component to reduce the disturbance effects.

1. State-Space Representation of a DC Motor

A DC motor can be characterized by the following parameters:

```
param.Kt = 0.035; % Torque proportionality constant (Kg m2 /s2/A) (N.m/A)
param.Ke = 0.035; % Back electromotive proportionality constant (V/rad/s)
param.L = 5e-3; % Electric inductance (H)
param.R = 0.1; % Terminal resistance (Ohm)
param.J = 5e-4; % Rotor moment of inertia (kg.m2)
param.b = 1e-06; % Rotor viscous friction constant (Kg.m2/s) (N.m.s)
```

Notice that these parameters differ from the one used for the hardware Qnet DC Motor, and are chosen for the purpose of this lab experiments. The complete state of the DC Motor is represented by $x(t)$

$$x(t) = \begin{pmatrix} \theta(t) \\ \omega(t) \\ i(t) \end{pmatrix}$$

where $\theta(t)$ is the angular position of the rotor, $\omega(t)$ is the angular velocity, and $i(t)$ is the current. The state-space representation of the DC Motor can be written as.

$$\frac{d}{dt}x(t) = \begin{pmatrix} 0 & 1 & 0 \\ 0 & -\frac{b}{J} & \frac{K_t}{J} \\ 0 & -\frac{K_e}{L} & -\frac{R}{L} \end{pmatrix} x(t) + \begin{pmatrix} 0 \\ 0 \\ \frac{1}{L} \end{pmatrix} v(t)$$

We are interested on controlling the angular position $\theta(t) = (1 \quad 0 \quad 0)x(t)$ to track a step input of 90 degrees.

The state-space representation of the DC Motor can be written as.

```
A = [ ...
    0, 1, 0; ...
    0, -param.b / param.J, param.Kt / param.J; ...
    0, -param.Ke / param.L, -param.R / param.L];
B = [ ...
    0; ...
    0; ...
    1 / param.L];
C = [1, 0, 0];
D = 0;
```

2. Position Control with Output Feedback

2.1 Full state feedback

In this section we assume that the state $x(t)$ is available at any time. Thus, we will design a state feedback controller of the form:

$$v(t) = -Kx(t) + Nr(t)$$

<<fig_lab07_full_state_feedback.png>>

Question 1 (0.5 marks)

Is the system controllable in the proposed state representation?

```
G = ss(A, B, C, D);
Gs = tf(G);


controllabilityMatrix = ctrb(A, B)
```

```
controllabilityMatrix = 3×3
10^5 ×
         0         0    0.1400
         0    0.1400   -2.8003
    0.0020   -0.0400   -0.1800
```

```
if det(controllabilityMatrix) ~= 0
    disp('The system is controllable')
else
    disp('The system is not controllable')
end
```

```
The system is controllable
```

Question 2 (2 marks)

1. Design the feedback gain K such that the state feedback places the closed-loop poles at the following locations: $-15 \pm 15j$, and -150.

2. Find the value of the pre-compensator N that will make the steady state error 0.
3. Complete the code to test the designed state-feedback controller.

```
p1 = -15 + 15j;
p2 = -15 - 15j;
p3 = -150;

Kpoles = [p1, p2, p3]
```

```
Kpoles = 1×3 complex
10^2 ×
  -0.1500 + 0.1500i  -0.1500 - 0.1500i  -1.5000 + 0.0000i
```

```
K = place(A, B, Kpoles)
```

```
K = 1×3
    4.8214    0.3185    0.8000
```

```
A1 = A - B * K;
Gc1 = ss(A1, B, C, D);
dcGainCL = dcgain(tf(Gc1));

N = 1/dcGainCL
```

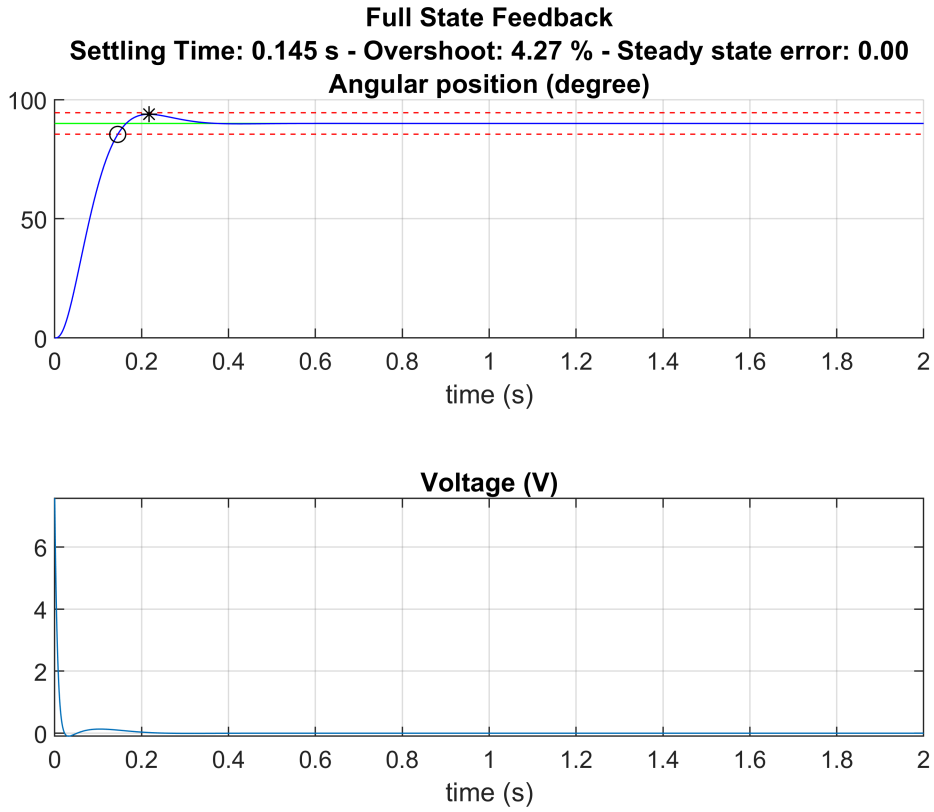
```
N = 4.8214
```

```
dt = 1e-3; % sampling time
t = 0:dt:2; % time signal
r = 90 * pi / 180 * ones(size(t)); % reference signal


Hcl = Gc1;


w = lsim(Hcl, N*r, t)'; % simulate the output
v = -K * lsim(ss(Hcl.A, Hcl.B, eye(3), 0), N*r, t)' + N * r; % simulate the command

customStep(sprintf('Full State Feedback'), t, w, r, v)
```



2.2 Output feedback with state observer

We assume that the state can't be fully measured and only the angular position can be measured. We will design a Luenberger observer based on the known model and the output $\theta(t)$. Recall that the equations of Luenberger observer are:

$$\frac{d}{dt}\hat{x}(t) = A\hat{x}(t) + Bv(t) + L(Cx(t) - C\hat{x}(t))$$

where L is the Luenberger gain.

<<fig_lab07_output_feedback.png>>

Question 3 (0.5 marks)

Is the system observable in the proposed state representation?

```
observabilityMatrix = obsv(A, C);


if det(observabilityMatrix) ~= 0
    disp('The system is observable')
else
    disp('The system is not observable')
end
```

```
The system is observable
```

Question 4 (1 marks)

Design a Luenberger state estimator such that the eigenvalues of the observer are $-10 \pm 10j$, and -100.

```
Lpoles = [-10+10j, -10-10j, -100];

L = place(A', C', Lpoles)'
```

```
L = 3×1
   99.9980
 -290.2000
 -331.4146
```

The state space representation of the output feedback system with a state observer can be represented by the augmented state:

$$x_{aug}(t) = \begin{pmatrix} x(t) \\ \hat{x}(t) \end{pmatrix}$$

and the feedback law is:

$$v(t) = -K\hat{x}(t) + Nr$$

With these notations, we can write the augmented state space equations as a function of A, B, C, K, and L

$$\frac{d}{dt}x_{aug}(t) = \begin{pmatrix} A & -BK \\ LC & A - LC - BK \end{pmatrix} x_{aug}(t) + \begin{pmatrix} B \\ B \end{pmatrix} v(t)$$

Question 5 (1.5 marks)

1. Write the augmented state-space representation matrices A_{aug} and B_{aug}.
2. Complete the code to simulate the response of output feedback system to follow a step input (use the same feedback gain N, and K from section 1.1).

```
% 1 -

Aaug = [A, -B*K; L*C, A-L*C-B*K];

Baug = [B; B];


% 2 -
Caug = [1, 0, 0, 0, 0, 0];

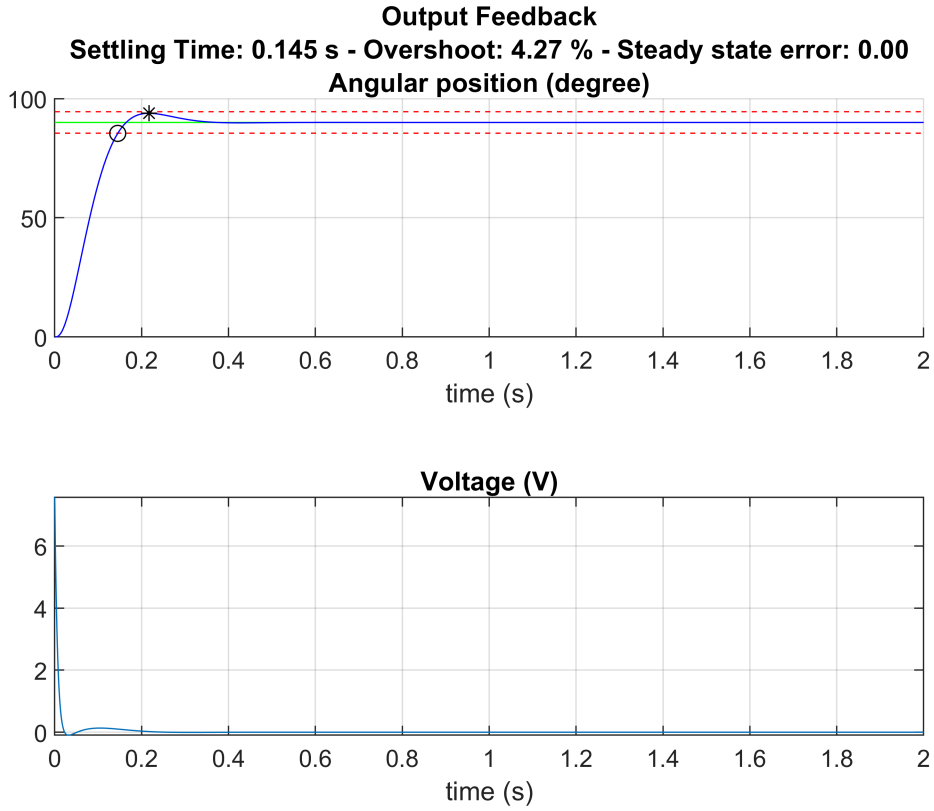

% define the augmented closed loop system
Hcl = ss(Aaug, Baug, Caug, 0);

% simulate the output
```

```
w = lsim(Hcl, N*r, t)';

w = w(:)'; % make the output a row array
v = -K * lsim(ss(Hcl.A, Hcl.B, [zeros(3), eye(3)], 0), N*r, t)' + N * r; % simulate the command

customStep(sprintf('Output Feedback'), t, w, r, v)
```



Question 6 (1 marks)

Let's change the Luenberger gain such as the observer dynamic is 10 times slower. re-run the experiment. What do you observe? Justify your answer.

```
Lnewpoles = 0.1 * Lpoles;

Lnew = place(A', C', Lnewpoles)';

K = place(A, B, Kpoles);

Aaug = [A, -B*K; Lnew*C, A-Lnew*C-B*K];

Baug = [B; B];

Caug = [1, 0, 0, 0, 0, 0];

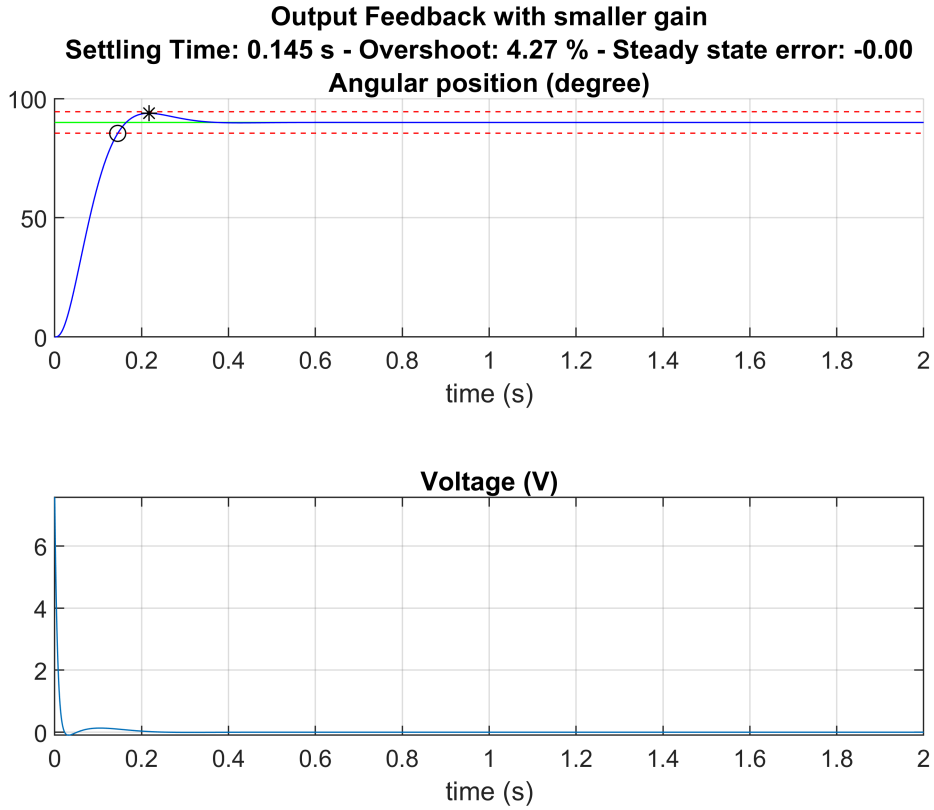
% define the closed loop system
Hcl = ss(Aaug, Baug, Caug, 0);

% simulate the output
```

```
w = lsim(Hcl, N*r, t)';


w = w(:)'; % make the output a row array
v = -K * lsim(ss(Hcl.A, Hcl.B, [zeros(3), eye(3)], 0), N*r, t)' + N * r; % simulate the comman

customStep(sprintf('Output Feedback with smaller gain'), t, w, r, v)
```




```
% We can observe that all the graphs in question 2, 5, and 6
% are the same. They have the same settling time of 0.145s,
% OS = 4.27% and a steady state error of 0. The angular position
% is also stable and constant at the end.

% We can see that the observer being faster or slower (by changing L)
% doesn't affect the output feedback of the system
```

3. Effects of wrong model

As seen in section 2 the importance of the Luenberger estimator will only be visible in case the dynamics of the model are not completely known. For example, in this section we assume that in the real DC Motor the inertia is 3 times what we initially thought. Which means that the true dynamics of the DC Motor are:

<<fig_lab07_estimator_wrong_model.png>>

```
Jreal = 3 * param.J;

Areal = [ ...
```

```
    0, 1, 0; ...
    0, -param.b / Jreal, param.Kt / Jreal; ...
    0, -param.Ke / param.L, -param.R / param.L];
```

For the purpose of this experiment we assume that we don't know about the true A_{real} and we only use A in the estimator dynamics.

Question 7 (1.5 marks)

1. Re-run the experiment using the same gains N, K, and L. But this time replace the transition matrix of the state space model of the DC Motor by A_{real}. What do you observe?
2. Change the design of L such as the observer is 5 times faster. What do you observe?

```
% 1
Aaug = [Areal, -B*K; L*C, Areal-L*C-B*K];

Baug = [B; B];

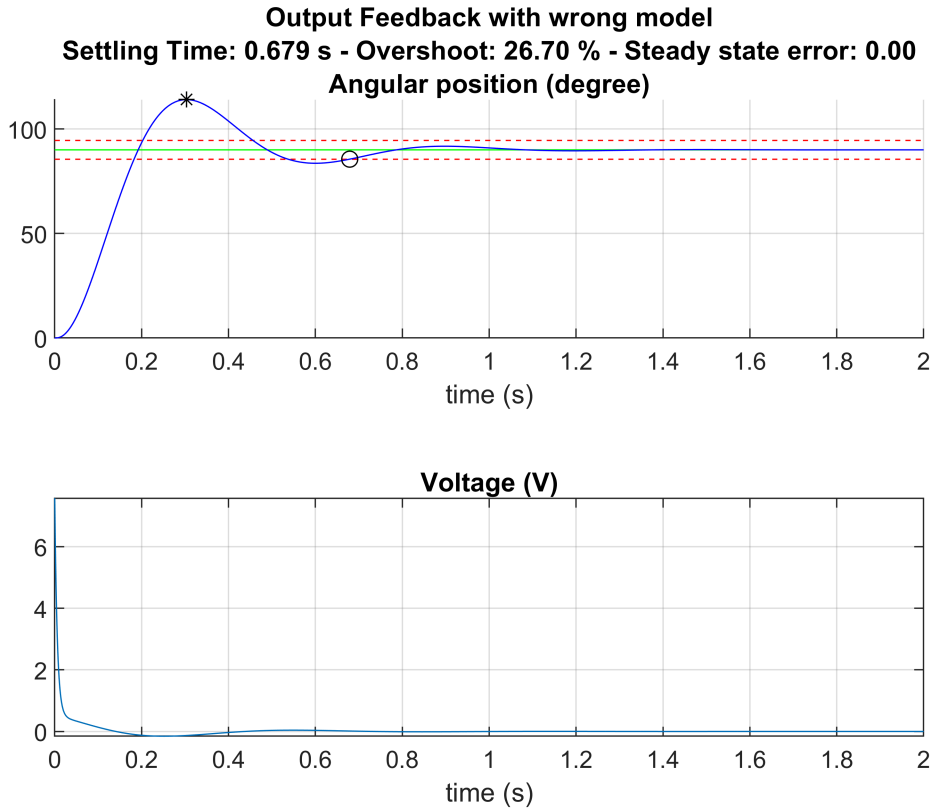
Caug = [1, 0, 0, 0, 0, 0];

% define the closed loop system
Hcl = ss(Aaug, Baug, Caug, 0);

% simulate the output
w = lsim(Hcl, N*r, t)';


w = w(:)'; % make the output a row array
v = -K * lsim(ss(Hcl.A, Hcl.B, [zeros(3), eye(3)], 0), N*r, t)' + N * r; % simulate the commanc

customStep(sprintf('Output Feedback with wrong model'), t, w, r, v)
```




```
% We can observe a higher overshoot percentage (26.70% vs 4.27%) and we can
% also see that the settling time has increased. We can thus say
% that with higher inertia comes higher overshoot and larger settling time.
```

```
% 2 -

Lnewpoles = 5 * Lpoles;

Lnew = place(A', C', Lnewpoles)';

K = place(A, B, Kpoles);

Aaug = [Areal, -B*K; Lnew*C, Areal-Lnew*C-B*K];

Baug =  [B;B];

Caug = [1, 0, 0, 0, 0, 0];

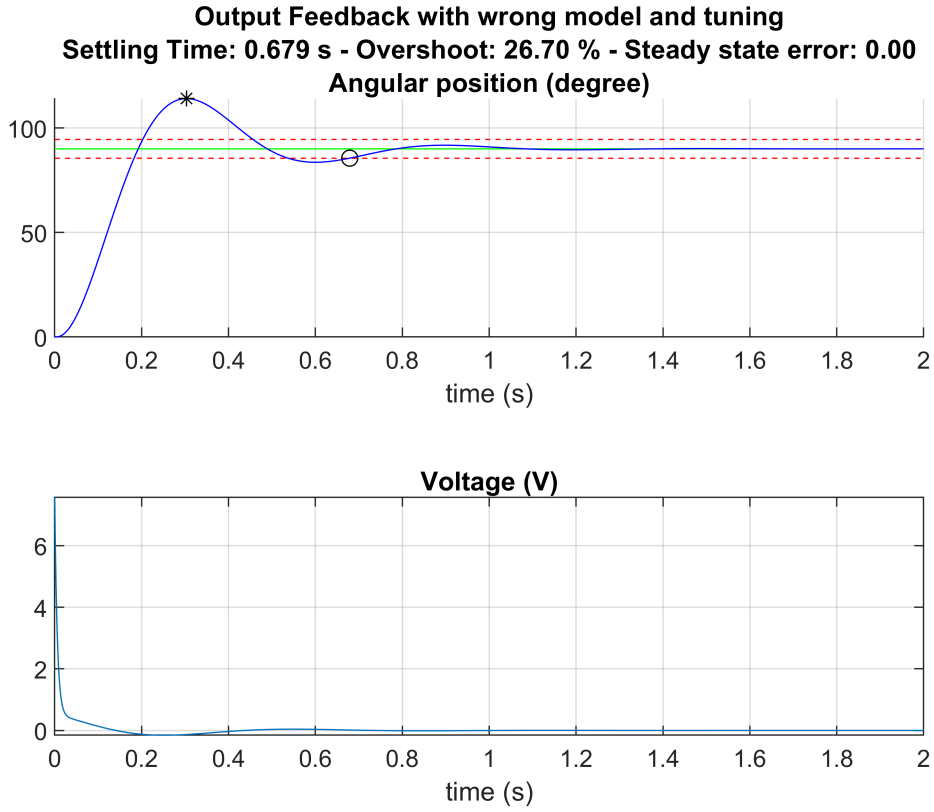
% define the closed loop system
Hcl = ss(Aaug, Baug, Caug, 0);

% simulate the output
w = lsim(Hcl, N*r, t)';


w = w(:)'; % make the output a row array
```

```
v = -K * lsim(ss(Hcl.A, Hcl.B, [zeros(3), eye(3)], 0), N*r, t)' + N * r; % simulate the command

customStep(sprintf('Output Feedback with wrong model and tuning'), t, w, r, v)
```




```
% The observer being faster or slower (by changing L)
% doesn't affect the output feedback of the system
```

4. Disturbance and Integral Design

4.1 Experiment with an additive disturbance

In this section we want to study the effects of a voltage disturbance on the state feedback controller. For this, we will simulate the effects of an additive disturbance into the voltage command of value $d = 4V$ at time = 1s.

To simplify the problem, we will consider that the whole state is available, that we know the exact model of the DC Motor (same in section 1), and that we have designed a full feedback controller K, and N.

Question 8 (0 marks)

What do you expect to happen to the angular position of the DC motor after applying a voltage command $d = 4V$ at time = 1s?

```
% There will be the superposition of another response starting at 1s
% So the angular position should increase to a higher value and
% settle eventually with a higher the steady state error.
```

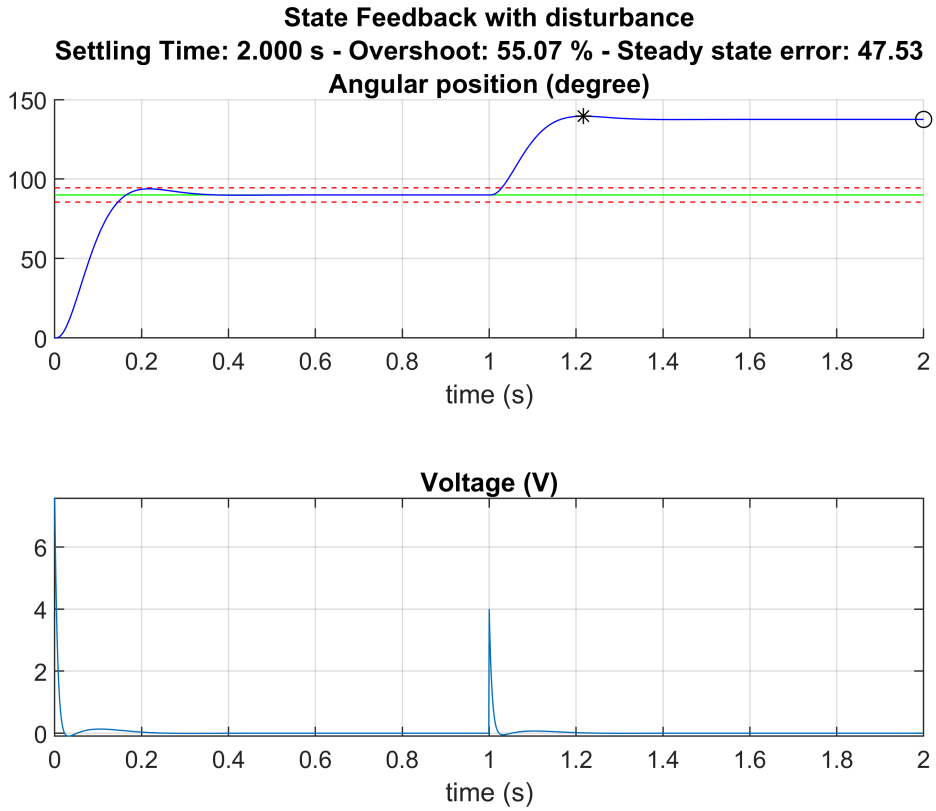
Let's simulate the result of adding such a disturbance in the closed-loop system

```
K = place(A, B, Kpoles);

d = zeros(size(t));
d(t >= 1) = 4; % disturbance signal

Hcl = ss(A-B*K, B, C, 0); % define the closed loop system
w = lsim(Hcl, N*r+d, t)'; % simulate the output
v = -K * lsim(ss(Hcl.A, Hcl.B, eye(3), 0), N*r+d, t)' + N * r + d; % simulate the command

customStep(sprintf('State Feedback with disturbance'), t, w, r, v)
```



Question 9 (0.5 marks)

What is the steady state error? What is the closed-loop system type?

```
% ss error
steadyStateError = 47.53
```

```
steadyStateError = 47.5300
```

```
% Because the steady state error is a non zero constant with step input,
% our closed loop system can be identified as TYPE0.
```

4.2 state feedback with integral term

We can add an integral term to the feedback controller to increase the type of the system. The integral equation is

$$x_n(t) = r(t) - Cx(t)$$

The augmented system is

$$x_{aug}(t) = \begin{pmatrix} x(t) \\ x_n(t) \end{pmatrix}$$

and the feedback law is:

$$v(t) = -Kx(t) + K_e x_n(t)$$

The equivalent system can be seen as a closed-loop system where the open-loop dynamics is defined by

$$\widetilde{A} = \begin{pmatrix} A & 0 \\ -C & 0 \end{pmatrix}$$

$$\widetilde{B} = \begin{pmatrix} B \\ 0 \end{pmatrix}$$

and a unity feedback

$$v(t) = -\begin{pmatrix} K & -K_e \end{pmatrix} x_{aug}(t)$$

Question 10 (1.5 marks)

1. Design K and Ke such as the poles of the feedback system are: $-15 \pm 15j$, and $-150 \pm 150j$.
2. Complete the code to run the simulation.
3. What do you observe?

```
% 1 - use place in the augmented closed-loop with integral system
p1d = -15 + 15i;
p2d = -15 - 15i;
p3d = -150 * (1 + 1i);
p4d = -150 * (1 - 1i);

Atilde = [A, zeros(3, 1); -C, 0];
Btilde = [B; 0];
Kall = place(Atilde, Btilde, [p1d, p2d, p3d, p4d])
```

```
Kall = 1×4
10^3 ×
    0.1061    0.0039    0.0015   -1.4464
```

```
K = Kall(1, 1:3) % Gain of the 1st 3 states
```

```
K = 1×3
  106.0714    3.8542    1.5500
```

```
Ke = -Kall(1, 4) % The gain not associated with first 3 states
```

```
Ke = 1.4464e+03
```

```
% 2 - Run the experiment

d = zeros(size(t));
d(t >= 1) = 4; % disturbance signal

% define the augmented closed-loop system

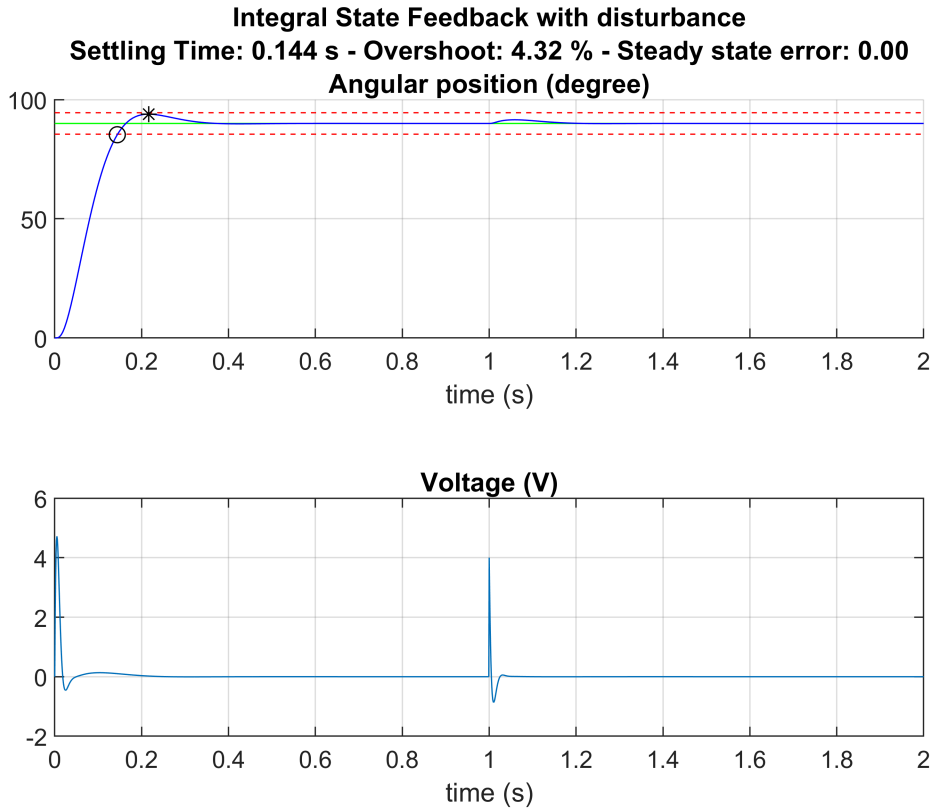
AtildeCl = Atilde - Btilde * Kall
```

```
AtildeCl = 4×4
10^5 ×
         0    0.0000         0         0
         0   -0.0000    0.0007         0
   -0.2121   -0.0078   -0.0033    2.8929
   -0.0000         0         0         0
```

```
% simulate the output
yr = lsim(ss(AtildeCl, [0; 0; 0; 1], [1, 0, 0, 0], 0), r, t)'; %response to reference
yd = lsim(ss(AtildeCl, Btilde, [1, 0, 0, 0], 0), d, t)'; %response to disturbance
w = yr + yd;

v = -[K, -Ke] * lsim(ss(AtildeCl, [[0; 0; 0; 1], Btilde], eye(4), 0), [r; d], t)' + d; % simula

customStep(sprintf('Integral State Feedback with disturbance'), t, w, r, v)
```




```
% 3 -
% The same 4V disturbance slightly changes the angular position,
% which quickly returns to the original steady-state value without the same
% "permanent" gain as before
```

5. Useful Functions

5.1 Custom step response

```
function customStep(figTitle, t, w, r, v)
% calculate response characteristics and plot results
settlingTimeIdx = find(round((w - 0.95 * r)*180/pi, 1) <= 0.05 | round((w - 1.05 * r)*180/pi, 2
[~, overshootIdx] = max(w);
steadyStateError = (180 / pi) * (w(end) - r(end));

figure;
clf;
hold on;
subplot(2, 1, 1)
hold on;
plot(t, (180 / pi)*0.95*r, '--r');
plot(t, (180 / pi)*1.05*r, '--r');
plot(t, (180 / pi)*r, 'g');
plot(t, (180 / pi)*w, 'b');
plot(t(settlingTimeIdx), (180 / pi)*w(settlingTimeIdx), 'ok')
plot(t(overshootIdx), (180 / pi)*w(overshootIdx), '*k')
```

```
grid on;
title(sprintf(['%s\n', ...
    'Settling Time: %4.3f s - Overshoot: %03.2f %% - Steady state error: %3.2f\n', ...
    'Angular position (degree)'], ...
    figTitle, ...
    t(settlingTimeIdx), ...
    100*(w(overshootIdx) - r(end))/r(end), ...
    steadyStateError))
xlabel('time (s)')
subplot(2, 1, 2)
plot(t, v);
grid on;
title('Voltage (V)');
xlabel('time (s)')
drawnow;
end
```